Documento de Justificación del Proyecto

Actividad de clase por equipo · Sprint 0 (Semana 1)

Propósito de la actividad

Este documento es un entregable obligatorio del Sprint 0. Antes de escribir una sola línea de código, el equipo debe **argumentar** (nivel Bloom: Evaluar) las decisiones que fundamentan su proyecto: por qué el problema exige concurrencia, qué tecnologías eligen y con qué criterio, cómo se organizan bajo Scrum y de qué forma su solución se vincula a un ODS. Es el filtro que garantiza que el equipo entiende su proyecto antes de construirlo.

Instrucciones: respondan cada sección en equipo. Las líneas grises son guía; sustitúyanlas por su contenido. El documento se entrega firmado por todos los integrantes al cierre de la Semana 1 y se sustenta oralmente en la Sprint Planning.

1. Datos de identificación

Nombre del proyecto:	Monitoreo ambiental en tiempo real
Equipo (nombre/número):	2
Product Owner:	Ríos Olivares César Antonio
Scrum Master:	Carmona Orea Lizbeth
Development Team:	Cucchi Villegas Samael; Delgadillo Escartín Eduardo Abraham Flores Teja Axel; Vázquez Rodríguez Rodrigo Enrique Gutiérrez Ponce Sebastián
Fecha:	17/09/2026

2. Justificación del problema y vínculo con el ODS

Describan el problema real que resuelven, a quién afecta, y con qué ODS y meta específica de la Agenda 2030 se alinea. No basta nombrar el ODS: expliquen cómo la solución contribuye a esa meta.

ODS seleccionado	Meta específica	Cómo contribuye el proyecto
11 y 13	Ingesta simultanea de N sensores y procesamiento paralelo de agregados con detección de umbrales	Contribuye a que los usuarios tengan una aplicación para ver el clima en tiempo real y también a proporcionar infraestructura de datos necesaria para crear ciudades más sostenibles. Al monitorear continuamente variables como la calidad del aire, las emisiones de gases o las anomalías de temperatura,

ODS seleccionado	Meta específica	Cómo contribuye el proyecto
		permite a las comunidades y autoridades tomar medidas preventivas contra la contaminación y gestionar políticas ambientales basadas en datos exactos.

3. Justificación de la concurrencia

Sección crítica. Un proyecto que solo *usa* hilos no basta; deben demostrar por qué los *necesitan*.

3.1 Escenario de condición de carrera

Describan un escenario concreto donde dos o más procesos acceden simultáneamente a un recurso compartido y, sin control, producirían un resultado incorrecto (dos usuarios reservan el mismo turno, dos reclaman el mismo lote, etc.). Este escenario es la razón de ser de su concurrencia.

Descripción del escenario:

Múltiples hilos concurrentes, cada uno escuchando la entrada de un sensor ambiental distinto (ej. calidad del aire, temperatura), necesitan actualizar simultáneamente un mismo acumulador global en memoria que calcula el promedio de contaminación para evaluar si se supera un umbral crítico.

Si el hilo del Sensor A y el hilo del Sensor B leen el valor actual del acumulador exactamente en el mismo ciclo de procesador, ambos incrementan su propia lectura, y luego escriben el resultado de vuelta en la memoria compartida, una de las escrituras sobrescribirá a la otra.

Resultado incorrecto: Se pierden los datos reales de uno o más sensores de forma silenciosa. El cálculo del promedio ambiental será artificialmente bajo y el sistema fallará en disparar la alerta de umbral de toxicidad a tiempo.

3.2 Recurso compartido y mecanismo de sincronización

Recurso compartido	Mecanismo elegido (mutex/semáforo/monitor/cola)	Por qué ese mecanismo
Variable global acumuladora (para calcular promedios en tiempo real de los sensores). Búfer de datos entrantes (cola donde los sensores dejan las lecturas antes del análisis).	Mutex y semáforos	Garantiza que solo un hilo a la vez pueda bloquear la variable, leerla, sumarle el nuevo dato del sensor y escribirla, evitando la sobreescritura cruzada. Es la solución de menor sobrecarga (overhead) para operaciones matemáticas rápidas sobre una variable simple. :Los sensores son los "productores" y los hilos de cálculo son los "consumidores". Los semáforos evitan el desbordamiento de memoria si llegan demasiados datos de golpe, y bloquean a los consumidores sin desperdiciar ciclos de CPU cuando la cola está vacía.

3.3 Modelo de concurrencia

¿Hilos, procesos, async? ¿Cuántos trabajadores concurrentes esperan? ¿Patrón productor-consumidor, pool de hilos, matching paralelo? Justifiquen.

Modelo: Patrón **Productor-Consumidor** apoyado en un **Pool de Hilos (Thread Pool)** y/o **Programación Asíncrona (async/await)**.

Justificación: Múltiples hilos "Productores" simulan o reciben constantemente datos de decenas de sensores IoT en red. Múltiples hilos "Consumidores" toman las métricas de la cola, aplican validaciones de umbrales y persisten la información en la base de datos sin bloquear el canal de entrada de nuevos eventos.

---

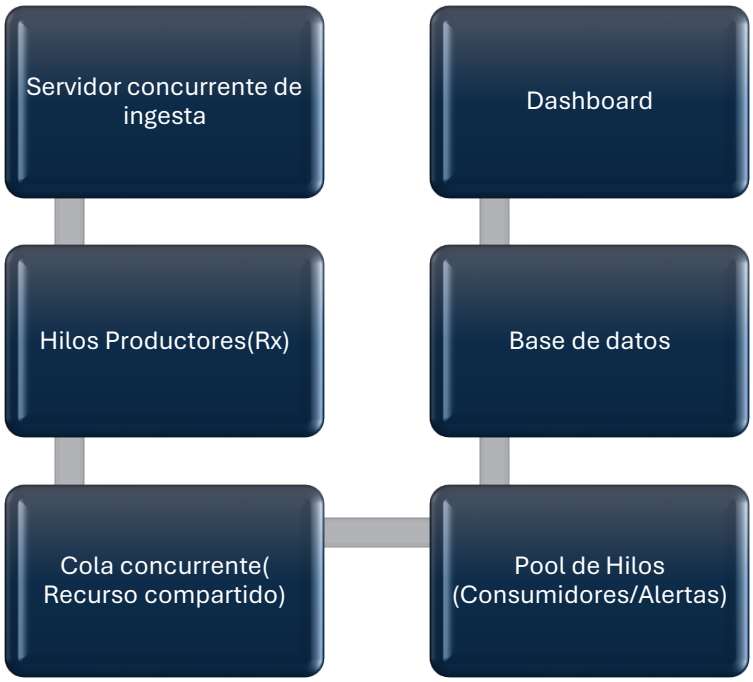
4. Selección de tecnologías

Por cada pilar, indiquen la tecnología elegida y el criterio de selección (dominio del equipo, soporte de concurrencia, ecosistema, licencia, rendimiento). No se evalúa la moda de la herramienta sino la calidad del argumento.

Pilar	Tecnología elegida	Justificación de la elección
Lenguaje / concurrencia	C++	Ofrece estructuras robustas, ideales para el procesamiento de concurrente y multihilo
Base de datos	MySQL	Soporta transacciones ACID y esta optimizada para la escritura rápida y simultanea de series temporales
Servicio web / móvil	FastAPI	Permite enviar actualizaciones concurrentes en vivo
Redes (si aplica)	MQTT	Es liviano, desacoplado y diseñado específicamente para el envió de concurrente
Herramientas (tablero, repo, pruebas)	GitHub	Facilitan el control de versiones y la gestión de tableros Scrum

5. Arquitectura de la solución

Inserten aquí el diagrama de arquitectura (componentes, flujo de datos, dónde ocurre la concurrencia y dónde la persistencia). Debajo, descríbanlo en un párrafo.



---

6. Organización del equipo según Scrum

Asignen los roles y describan la responsabilidad de cada integrante. Recuerden: Product Owner y Scrum Master son roles, no jerarquías; todos programan salvo el PO cuando el equipo lo acuerde.

Rol Scrum	Integrante	Responsabilidad principal en este proyecto
Product Owner	Ríos Olivares César Antonio	Prioriza el backlog, define el valor y el vínculo con el ODS.
Scrum Master	Carmona Orea Lizbeth	Facilita ceremonias, remueve impedimentos, cuida el proceso.
Dev Team (BD)	Delgadillo Escartín Eduardo Abraham	Diseño e integridad de datos y control de concurrencia en datos.
Dev Team (núcleo concurrente)	Gutiérrez Ponce Sebastián	Implementación de hilos y sincronización.
Dev Team (servicio/red)	Gutiérrez Ponce Sebastián	API/app y comunicación cliente-servidor.

7. Product Backlog inicial

Redacten al menos 6 historias de usuario con el formato: «Como <rol>, quiero <acción> para <beneficio>». Asignen prioridad (Alta/Media/Baja) y una estimación relativa (puntos de historia).

#	Historia de usuario	Prioridad	Puntos
1	,"Como sistema central, quiero recibir lecturas de múltiples sensores simultáneamente usando hilos para no perder datos durante picos de información ambiental. (Núcleo concurrente)"	Alta	8
2	Como analista ambiental, quiero que el sistema calcule promedios en paralelo para detectar si se supera un umbral de contaminación en tiempo real sin retrasos. (Núcleo concurrente)"	Alta	8
3	Como aplicación cliente, quiero conectarme a una API expuesta por el servidor para visualizar el estado actualizado del clima y la calidad del aire. (API / Red)",	Alta	5
4	"Como administrador de red, quiero que la comunicación entre los sensores y el servidor sea constante y ligera para mantener el flujo de datos sin saturar el ancho de banda. (API / Red)"	Media	5
5	Como auditor del sistema, quiero que cada lectura procesada se almacene de forma transaccional en la base de datos para evitar inconsistencias si dos sensores envían datos al mismo milisegundo. (BD)"	Media	5
6	Como ciudadano, quiero recibir alertas automáticas generadas por el servidor para tomar medidas preventivas si el ambiente es tóxico. (Integración General)"	Baja	6

8. Definición de Terminado (Definition of Done)

Acuerden qué significa que una historia esté «terminada» en este proyecto. Ejemplos abajo; ajústenlos.

- El código está versionado en el repositorio y revisado por al menos un compañero.
- La funcionalidad pasa las pruebas de concurrencia (sin condiciones de carrera observables).
- La operación sobre base de datos es transaccional e íntegra.
- La historia está documentada y demostrable en la Sprint Review.

Control de Versiones y Revisión Cruzada: El código está integrado en la rama principal del repositorio, se ejecuta sin errores en el entorno de desarrollo (por ejemplo, al correr desde la terminal en Linux), y los cambios fueron revisados por al menos un compañero de un rol distinto (ej. un cambio en la API validado por el equipo del núcleo concurrente).

Pruebas de Concurrencia Exitosas: La funcionalidad soporta la carga de trabajo simulando N sensores simultáneos mediante múltiples hilos, demostrando a nivel de sistema que no existen condiciones de carrera (race conditions) ni bloqueos (deadlocks) al calcular los agregados.

Transaccionalidad e Integridad de BD: Toda escritura en la base de datos mantiene la atomicidad; si varios hilos intentan registrar datos de sensores o alertas de umbral en el mismo milisegundo, la información se guarda correctamente sin corromperse ni sobrescribirse.

Comunicación Cliente-Servidor Validada: Las rutas de la API o los canales de red responden correctamente a las peticiones del cliente de forma asíncrona, comprobando que el envío de datos no bloquea la escucha del monitor principal.

Demostrable y Documentada: La historia cuenta con documentación técnica (ej. rutas de la red detalladas o justificación de los cerrojos utilizados) y la funcionalidad está lista para ejecutarse en vivo durante la Sprint Review.

¿Para las pruebas de la API y los hilos concurrentes, tienen pensado montar y ejecutar todo el sistema directamente en alguna distribución de Ubuntu, o lo probarán de forma local sobre Windows antes de integrarlo?

9. Compromiso del equipo

Los integrantes firmamos este documento asumiendo las decisiones y roles aquí declarados para el desarrollo del proyecto durante las 9 semanas.

Carmona Orea Lizbeth Nombre y firma	Flores Teja Axel Nombre y firma
Vázquez Rodríguez Rodrigo Enrique Nombre y firma	Samael Cucchi Villegas Nombre y firma

Delgadillo Escartín Eduardo Abraham

10. Rúbrica de evaluación de este documento

Criterio	Insuficiente	Suficiente	Competente	Destacado
Justificación de la concurrencia	No plantea condición de carrera.	La plantea, pero es vaga o irreal.	Escenario concreto y correcto con recurso y mecanismo.	Además justifica el mecanismo frente a alternativas.
Selección de tecnologías	Sin criterio, solo lista.	Criterio débil o genérico.	Cada elección justificada por su aporte al proyecto.	Argumenta trade-offs entre opciones.
Roles Scrum	Roles sin responsabilidades.	Roles asignados, responsabilidades vagas.	Roles y responsabilidades claras y coherentes.	Refleja comprensión madura del marco Scrum.
Vínculo con el ODS	ODS decorativo.	Vínculo declarado sin justificar.	ODS y meta justificados con la solución.	Explica contribución medible a la meta.

Actualización de integrantes del equipo

Monitoreo Ambiental en Tiempo Real - Equipo 2

Complemento documental elaborado el 22 de octubre de 2026.

El documento original conserva sin cambios los roles de Product Owner y Scrum Master.
Este complemento actualiza las asignaciones del Dev Team indicadas en la sección 6.
Las firmas originales se conservan.
Product Owner: Ríos Olivares César Antonio (sin cambios).
Scrum Master: Carmona Orea Lizbeth (sin cambios).
Dev Team (Base de Datos): Delgadillo Escartín Eduardo Abraham.
Dev Team (Núcleo concurrente): Gutiérrez Ponce Sebastián.
Dev Team (Servicio/Red): Gutiérrez Ponce Sebastián.